Nanyang Technological University

College of Computing and Data Science

SC2002 Object-Oriented Design & Programming

Restaurant Rush

Group Assignment Report

Lab group: FCS1
Group: Group X
Language: Java (JDK 17 or later), command-line interface
Repository: https://github.com/SeraphinaXYX/SC2002-Restaurant-Rush
Stage tags: `stage-1`, `stage-2`, `stage-3`

Group Members

Name	Matric No.	Main contribution
Member A	U2XXXXXXX	Turn loop and game rules (`GameController`, `GameConfig`)
Member B	U2XXXXXXX	Domain model (`Customer`, `Order`, `Table`, `Restaurant`)
Member C	U2XXXXXXX	Staff classes, `ActionResult`; Stage 2 customers
Member D	U2XXXXXXX	Stage 3 (`Host`, `ComboMeal`, `Menu`, `HappyHour`)
Member E	U2XXXXXXX	CLI (`GameView`), UML diagrams, report, video

October 2026

Contents

1 Introduction **1**

2 UML Class Diagram **1**
2.1 Relationships 4
2.2 Encapsulation, Polymorphism and Composition 5

3 UML Sequence Diagram **6**
3.1 Chosen Scenario 6
3.2 Sequence Diagram 7
3.3 Notes on the Diagram 7
3.4 How the Manager's Choice Changes the Outcome 10

4 Design Discussion **10**
4.1 Key Abstractions and Responsibilities 10
4.2 Design Principles 12
4.3 Extensibility Walkthrough 13
4.4 Trade-offs 13
4.5 Alternative Design Considered: State Pattern for the Customer Lifecycle 13
4.6 How the Player's Choices Change the Result 14

5 GitHub Evidence and Stage Transitions **14**
5.1 Repository and Tags 14
5.2 Stage 1 → Stage 2 15
5.3 Stage 2 → Stage 3 16
5.4 What Stayed the Same, and How We Checked 17
5.5 How the Stage 1 Design Took the Later Requirements 17

6 Functional Evidence **18**
6.1 Summary of reproducible runs 18
6.2 Start-of-turn display 18
6.3 Invalid choices are explained and do not use up the task 19
6.4 READY, SERVED, READY_TO_PAY, PAID: the worked example from the brief 19
6.5 Stage 2: VIP and Critic behaviour 20
6.6 Stage 3: Host, ComboMeal and Happy Hour 20
6.7 Abandonment, Victory and Defeat 21

7 Reflections **22**
7.1 Who Did What 22
7.2 Problems We Hit 22
7.3 What We Would Do Differently 23
7.4 What We Learned 23

8 Video Timestamps **24**

List of Figures

1 Final class diagram (Stage 3). Plain getters and constructors that only copy their arguments are omitted. Relationships that are not drawn are listed in Table 2. . . 3
2 Sequence diagram, part 1 (turn 7): VIP arrival, Happy Hour activation, the Host seating the VIP, the Waiter taking the order at a locked price, and cooking. 8

3 Sequence diagram, part 2 (turns 8–10 and the result check): Happy Hour recovery, serving with validation, eating, payment, and the end-of-game check. Each alt/[else] pair is one alternative fragment. The values returned by the result check are the real ones from this run. 9

List of Tables

1 Base setting used in all three stages 1
2 Relationships in the final design 4
3 Timeline of the sequence-diagram scenario (Stage 3, seed 1) 6
4 The same seed-1 game with and without Happy Hour 10
5 Key abstractions and their responsibilities 11
6 Cost of likely future changes 13
7 Main design trade-offs 13
8 Contents of each tagged snapshot 15
9 Source files added and changed from Stage 1 to Stage 2 (157 lines added, 8 removed) 15
10 Source files added and changed from Stage 2 to Stage 3 (315 lines added, 14 removed) 16
11 Replaying earlier runs on the final build 17
12 Reproducible runs (all numbers taken from the final screen of each .out file) . . . 18
13 Ownership by member 22
14 Video segments 24

1 Introduction

Restaurant Rush is a turn-based restaurant management game played from the command line. The player is the manager. Each turn they decide what the Waiter and the Chef spend their single action on, while customers keep arriving and lose patience. Because staff time is the bottleneck, the interesting decisions are about priority: who gets seated first, which order the kitchen works on, and when to serve a finished dish instead of seating someone new.

Base setting

Every build of the game offers the base setting from the brief, so that runs can be reproduced with a fixed seed.

Table 1: Base setting used in all three stages

Setting	Value
Turns	18 (the game always plays all 18)
Tables	2
Staff	1 Waiter (one task per turn), 1 Chef (one preparation unit per turn), 1 Cashier (one payment per turn)
Arrivals	One customer at the start of turns 1, 4, 7, 10, 13 and 16
Menu	Salad $9.00 (1 unit), Burger $15.00 (1 unit), Pasta $18.00 (2 units)
Victory	At least 4 paid customers, revenue of at least $45.00, and average satisfaction of paid customers of at least 60

The three stages

We built the game in three runnable stages, each tagged in the repository. **Stage 1** (`stage-1`) is the complete core game with Regular customers, the three menu items, the Waiter, Chef and Cashier, and both endings. **Stage 2** (`stage-2`) adds VIP and Critic customers. Their behaviour lives in `VIPCustomer` and `CriticCustomer`, and `GameController` did not change at all between the two tags. **Stage 3** (`stage-3`) adds a Host who can seat one extra customer per turn, `ComboMeal` orders made of two menu items, and a one-off Happy Hour pricing policy. With seed 42, our recorded victory runs end with 5 paid customers and $63.00 (Stage 1), 4 paid and $53.10 (Stage 2), and 5 paid and $88.45 (Stage 3). The matching defeat runs end with nobody paid.

How this report is organised

Section 2 presents the final class diagram, drawn from the Stage 3 code (UML class diagram, 25 marks). Section 3 follows one full scenario as a sequence diagram: Happy Hour is switched on, a VIP arrives, orders, is served, eats and pays, and the result is checked (UML sequence diagram, 20 marks). Section 4 discusses responsibilities, the design principles that the code really reflects, trade-offs and an alternative we rejected (design consideration, 15 marks). Section 5 analyses the changes between the stage tags using the real diffs, and Section 6 shows transcripts from reproducible runs. Together they support the implementation and development-process criteria (20 marks each). Section 7 contains our reflections and Section 8 lists the video segments.

2 UML Class Diagram

Figure 1 is the class diagram of our final design. It was checked line by line against the `stage-3` source, so every class, field and operation in it exists in the code under the same name and with the

same parameters. To keep it readable we left out plain getters (such as `getId()` or `getStatus()`) and constructors that only copy their arguments into fields. A constructor is shown when it carries a design decision, for example `Menu(includeCombos)` or `Restaurant(tableCount)`.

Notation. We use standard UML 2 notation: `+` public, `-` private, `#` protected; *italic* operations are abstract and underlined members are static. Operations with no return type return `void`. A solid line with a hollow triangle is generalisation and a dashed one is interface realisation. A filled diamond is composition, a hollow diamond is aggregation, a solid line (with an open arrowhead when it is navigable in one direction only) is an association, and a dashed line with an open arrowhead is a dependency. The shaded panels group the classes into the boundary, control, staff and entity layers. A few associations and dependencies are not drawn because they would only add crossing lines, for example `GameController` → `Menu`; they are all listed in Table 2.

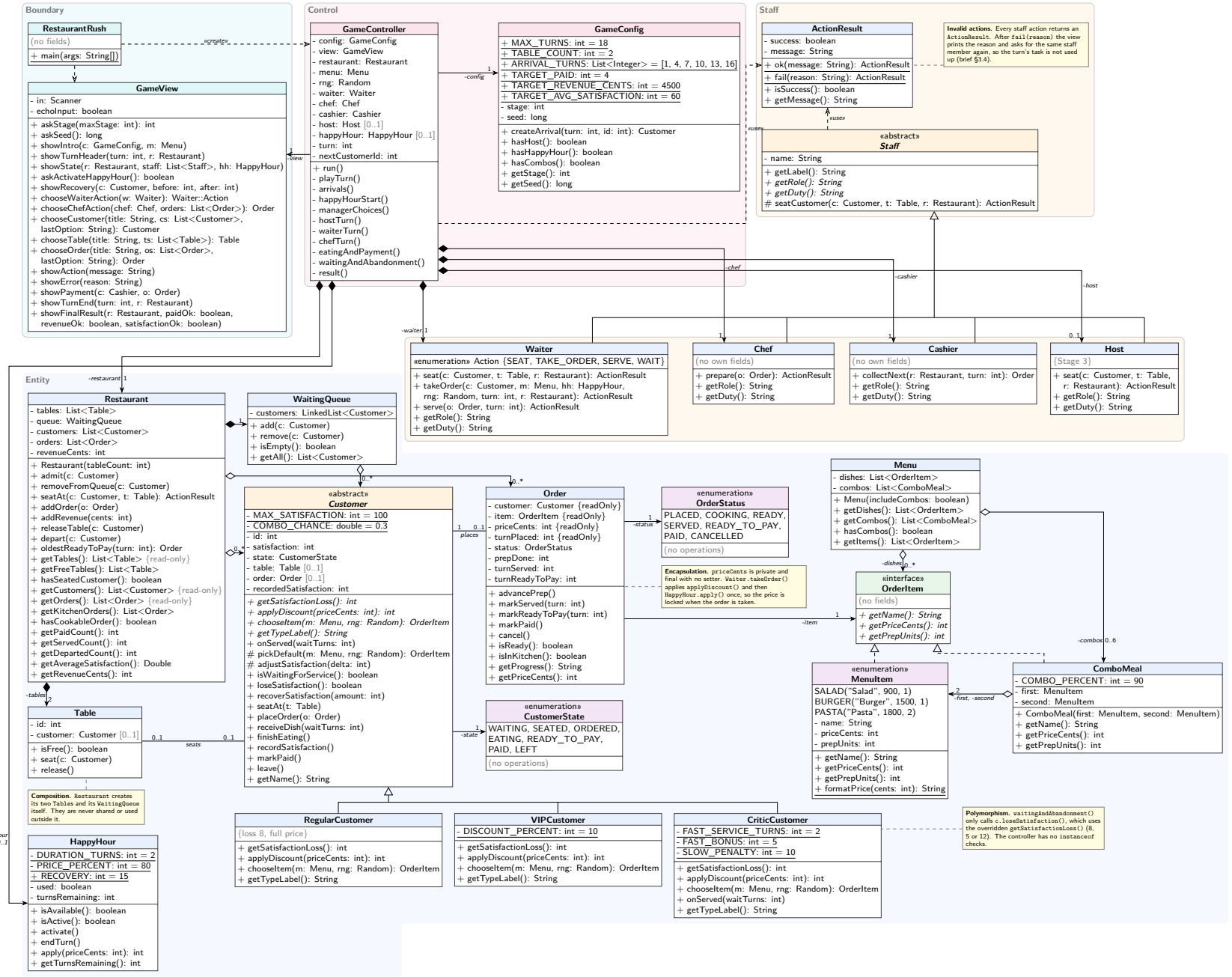


Figure 1: Final class diagram (Stage 3). Plain getters and constructors that only copy their arguments are omitted. Relationships that are not drawn are listed in Table 2.

2.1 Relationships

Table 2 lists every relationship in the final design, including the ones we left out of the figure to avoid crossing lines.

Table 2: Relationships in the final design

From	Kind	To (mult.)	Where it shows up in the code
RegularCustomer, VIPCustomer, CriticCustomer	generalisation	Customer	each overrides the four abstract operations; only the Critic overrides `onServed()`
Waiter, Chef, Cashier, Host	generalisation	Staff	each implements `getRole()` and `getDuty()`; Waiter and Host share `seatCustomer()`
MenuItem, ComboMeal	realisation	OrderItem	`Order` and `Chef` only see `OrderItem`
GameController	composition	Restaurant (1), Menu (1)	created in the controller's constructor
GameController	composition	Waiter, Chef, Cashier (1 each)	field initialisers in `GameController`
GameController	composition	Host (0..1), HappyHour (0..1)	created only when `config.hasHost()` / `hasHappyHour()`, otherwise `null` (Stages 1 and 2)
GameController	association	GameView (1), GameConfig (1)	passed into the constructor by `main()`
Restaurant	composition	Table (2), WaitingQueue (1)	created by `Restaurant` itself
Restaurant	aggregation	Customer (0..*), Order (0..*)	customers and orders are also referenced by tables, the queue and each other
WaitingQueue	aggregation	Customer (0..*)	a customer leaves the queue but keeps existing
Table	association	Customer (0..1 – 0..1)	`Table.customer` and `Customer.table`
Customer	association	Order (1 – 0..1)	`Order.customer` and `Customer.order`
Customer, Order	association	CustomerState, OrderStatus (1)	`state` and `status` fields
Order	association	OrderItem (1)	`Order.item`
Menu	aggregation	OrderItem (3), ComboMeal (0..6)	`dishes` and `combos`; six combos only when `includeCombos` is true
ComboMeal	aggregation	MenuItem (2)	`first`, `second`; enum constants are shared, so this is not composition

From	Kind	To (mult.)	Where it shows up in the code
Staff, Restaurant, GameController	dependency «use»	ActionResult	returned by `seat`, `takeOrder`, `serve`, `prepare` and `Restaurant.seatAt`; read by the controller
Waiter	dependency	Menu, HappyHour, Order	parameters of `takeOrder()` and `serve()`; `takeOrder()` creates the `Order`
Cashier	dependency	Restaurant, Order	`collectNext(r, turn)` returns the paid `Order` or `null`
Customer	dependency	Menu	parameter of `chooseItem()` and `pickDefault()`
GameConfig	dependency «create»	Customer subclasses	`createArrival()` instantiates them
GameView	dependency	Restaurant, Staff, HappyHour, Order, Table	read-only parameters of the display and choice methods
RestaurantRush	dependency «create»	GameController, GameView, GameConfig	`main()` creates and wires them

2.2 Encapsulation, Polymorphism and Composition

Encapsulation. All instance fields are private; the only public fields are `static final` constants (the base setting in `GameConfig` and `HappyHour.RECOVERY`). State changes go through methods that check the current state first. `Order.advancePrep()` only accepts an order that is `PLACED` or `COOKING` and turns it `READY` after its last unit. `Restaurant.seatAt()` refuses a customer who is not `WAITING` and a table that is already occupied, and returns the reason in an `ActionResult`. `Order.priceCents` is final and has no setter, which is how we meet the rule that a price is locked when the order is taken. `WaitingQueue.getAll()` and `Restaurant.getOrders()` return read-only lists, so `GameView` can print the state but cannot change it.

Polymorphism. The three customer types differ only through overridden methods: `getSatisfactionLoss()` (8, 5, 12), `applyDiscount()` (the VIP takes 10% off), `chooseItem()` (the Critic always picks the most expensive single dish, never a combo) and `onServed()` (the Critic gains 5 if served within two turns of ordering and loses 10 otherwise). `GameController` and `Waiter` only ever hold a `Customer` reference, and `Waiter.serve()` calls `receiveDish()`, which calls the overridden `onServed()`. The same idea applies to `OrderItem`: `Chef.prepare()` works on a `ComboMeal` exactly as on a `MenuItem`, because `Order.advancePrep()` only calls `getPrepUnits()`.

Composition and aggregation. `Restaurant` owns its tables and its waiting queue, and `GameController` owns the `Restaurant`, the `Menu`, the staff and the optional `Host` and `HappyHour`; none of these outlive their owner. We used aggregation where the parts are shared. Customers are referenced from the restaurant, the queue and a table at the same time, and a `ComboMeal` points to two `MenuItem` constants that every other order also uses.

3 UML Sequence Diagram

3.1 Chosen Scenario

The scenario is a real Stage 3 game in the base setting with seed 1. It is the committed run `test-runs/stage3-vip-happy-hour-seed1.txt`, so a TA can replay it exactly. At the start of turn 7 the manager switches on Happy Hour. In the same turn Customer 3, a VIP, arrives. The Host seats the VIP at Table 1, and the Waiter then takes the VIP's order while Happy Hour is active. The VIP picks a Burger, and the price is locked at

$$\$15.00 \times 0.9 \text{ (VIP)} \times 0.8 \text{ (Happy Hour)} = \$10.80.$$

The Chef finishes the Burger on turn 7. The Waiter serves it on turn 8, the VIP finishes eating on turn 9, and the Cashier collects $10.80 on turn 10. After turn 18 the controller runs the result check, which gives Victory with 5 paid customers, $73.80 revenue and an average satisfaction of 91.4.

The scenario covers each item that Section 5 of the brief lists for the sequence diagram:

- customer arrival (created by `GameConfig`) and the order (an `Order` created by the Waiter);
- the Waiter and Chef assignments chosen by the manager;
- the order going `READY` and then `SERVED`;
- eating, payment and the end-of-game result check;
- later-stage behaviour: VIP pricing (Stage 2), and the Host and Happy Hour (Stage 3).

Part 2 also shows what happens when the manager picks a choice that is not allowed: the Waiter returns a failed `ActionResult` with a reason and the manager is asked again. This can happen on any staff prompt; we draw it once, for serving.

Turn	Phase	What happens (from the real run)	VIP sat.
7	1 Arrivals	`GameConfig.createArrival(7, 3)` creates a `VIPCustomer`; `Restaurant.admit()` puts it in the queue.	100
	Start	Manager switches Happy Hour on (active turns 7 and 8). The VIP is already at 100, so recovery changes nothing.	100
	2–3 Staff	Host seats the VIP at Table 1. Waiter takes the order: Burger, list $15.00, charged $10.80. Chef prepares 1/1, so the order is `READY`.	100
	4 Payment	Cashier collects $15.00 from Regular #2 (not part of this scenario).	100
	5 Waiting	Ordered but not served: −5.	95
8	Start	Happy Hour recovery: 95 → 100.	100
	2–3 Staff	Waiter serves the Burger, which was already `READY` before the Waiter acted. It becomes `SERVED`; no more satisfaction loss. Happy Hour ends after this turn.	100
9	4 Eating	Cashier is called first and finds nobody to charge. The order, served on an earlier turn, then moves to `READY_TO_PAY`.	100
10	4 Payment	`Cashier.collectNext()`: `PAID`, revenue +$10.80, satisfaction 100 recorded, Table 1 released.	100
18	6 Result	5 paid, $73.80, average 91.4 against 4 / $45 / 60: Victory.	–

Table 3: Timeline of the sequence-diagram scenario (Stage 3, seed 1)

3.2 Sequence Diagram

We split the diagram into two parts so that it stays readable at A4 size. Part 1 (Figure 2) covers turn 7: arrival, Happy Hour, seating, ordering and cooking. Part 2 (Figure 3) covers turns 8–10 (recovery, serving, eating and payment) and the result check. Messages that only concern other customers, such as the payment from Regular #2 on turn 7, are left out. The notation we use is as follows:

- Solid arrows are synchronous calls and dashed arrows are returns.
- A message labelled «create» creates the object at the end of the arrow.
- Activation bars show which object currently has control.
- The loop fragment is the 18-turn game loop; the turn blocks only group the messages of one turn.
- opt and alt fragments carry their guards in square brackets.

Only `:GameView` talks to the manager. Every rule is checked inside `:GameController`, the staff objects or the domain objects. All message names are the real method names in the `stage-3` code.

3.3 Notes on the Diagram

Price locking. `Waiter.takeOrder()` works out the final price only once. It first calls `applyDiscount()` on the `VIPCustomer`, then `HappyHour.apply()`, which only discounts while Happy Hour is active. The result goes into the constructor of the new `Order`, and since `priceCents` is final it cannot change afterwards. When the Cashier collects on turn 10, Happy Hour has already ended, but the bill is still $10.80. This is how we follow the rule "the price is locked when the order is taken".

Object creation. Two objects are created during the scenario. `GameConfig.createArrival(7, 3)` decides which customer type arrives on turn 7 and creates the `VIPCustomer`; the controller only passes the returned object to `Restaurant.admit()`. The Waiter creates the `Order` inside `takeOrder()`, because that is the moment the price is fixed.

Host and Waiter in the same turn. `Host.seat()` and `Waiter.seat()` both delegate to `Staff.seatCustomer()`, which calls `Restaurant.seatAt()`; the diagram shows the call that `seatCustomer` makes. Because the Host seats the VIP, the Waiter's one task for turn 7 is still free, so the order can be taken in the same turn.

Polymorphic dispatch. The Waiter only knows it is talking to a `Customer`. `chooseItem`, `applyDiscount` and `onServed` are resolved at run time to the `VIPCustomer` versions; the VIP does not override `onServed`, so the inherited no-op runs. If a `CriticCustomer` were in the same place, the same messages would pick the most expensive dish and apply the speed bonus or penalty. Neither the controller nor the Waiter checks the customer's type.

Timing rules. The Waiter acts before the Chef in every turn, so the Burger finished on turn 7 can only be served on turn 8. Inside step 4 the Cashier is called before eating is updated. On turn 9 this means the Cashier finds nobody to charge and only afterwards does the order become `READY_TO_PAY`, so payment happens on turn 10. `SERVED` (8), `READY_TO_PAY` (9) and `PAID` (10) each fall on a different turn, as in the worked example in Section 3.3 of the brief.

Invalid choices. `serve()` returns an `ActionResult`. In the [else] branch the order is not `READY`, so the Waiter returns `ActionResult.fail` with a reason such as "Burger for VIP #3 is

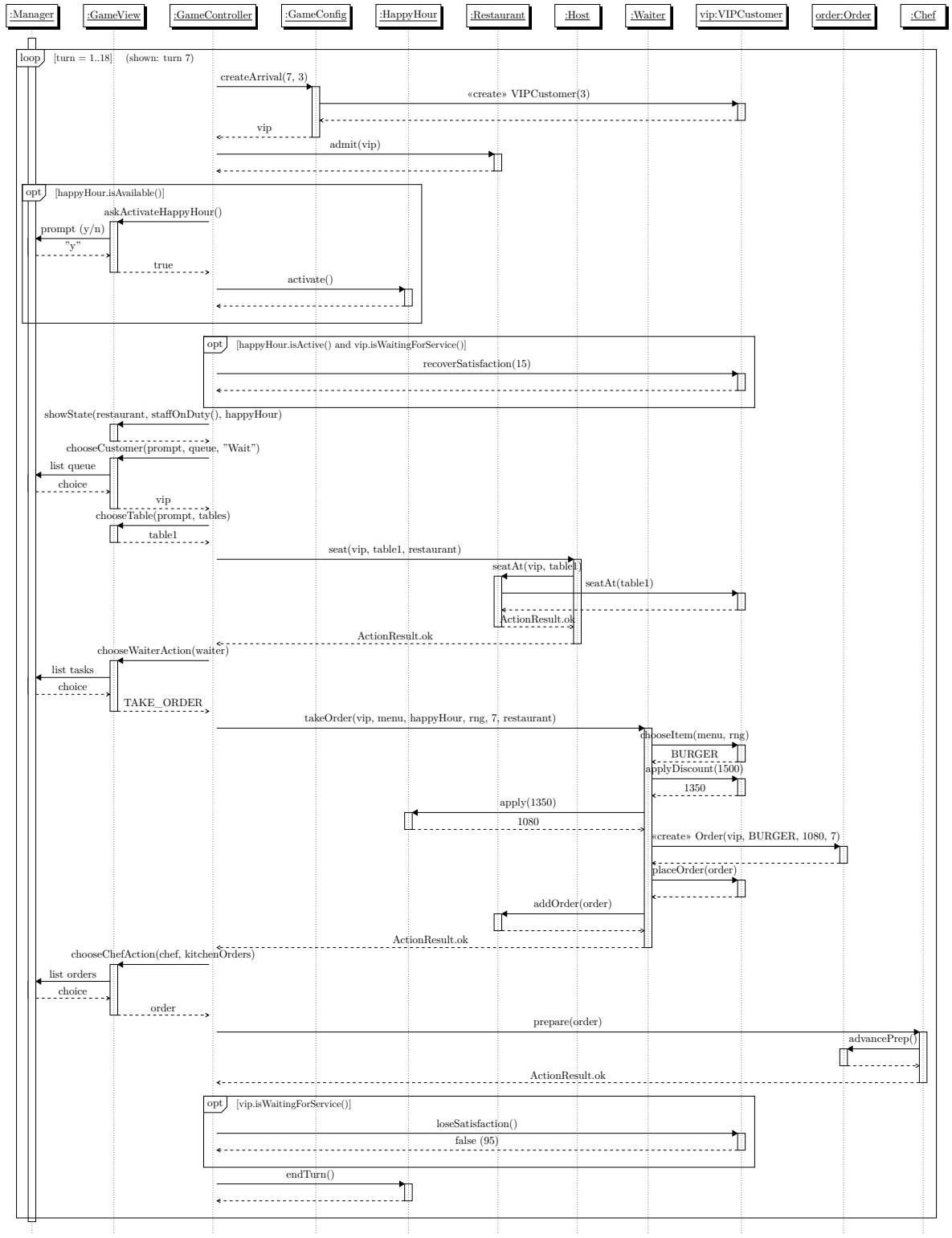


Figure 2: Sequence diagram, part 1 (turn 7): VIP arrival, Happy Hour activation, the Host seating the VIP, the Waiter taking the order at a locked price, and cooking.

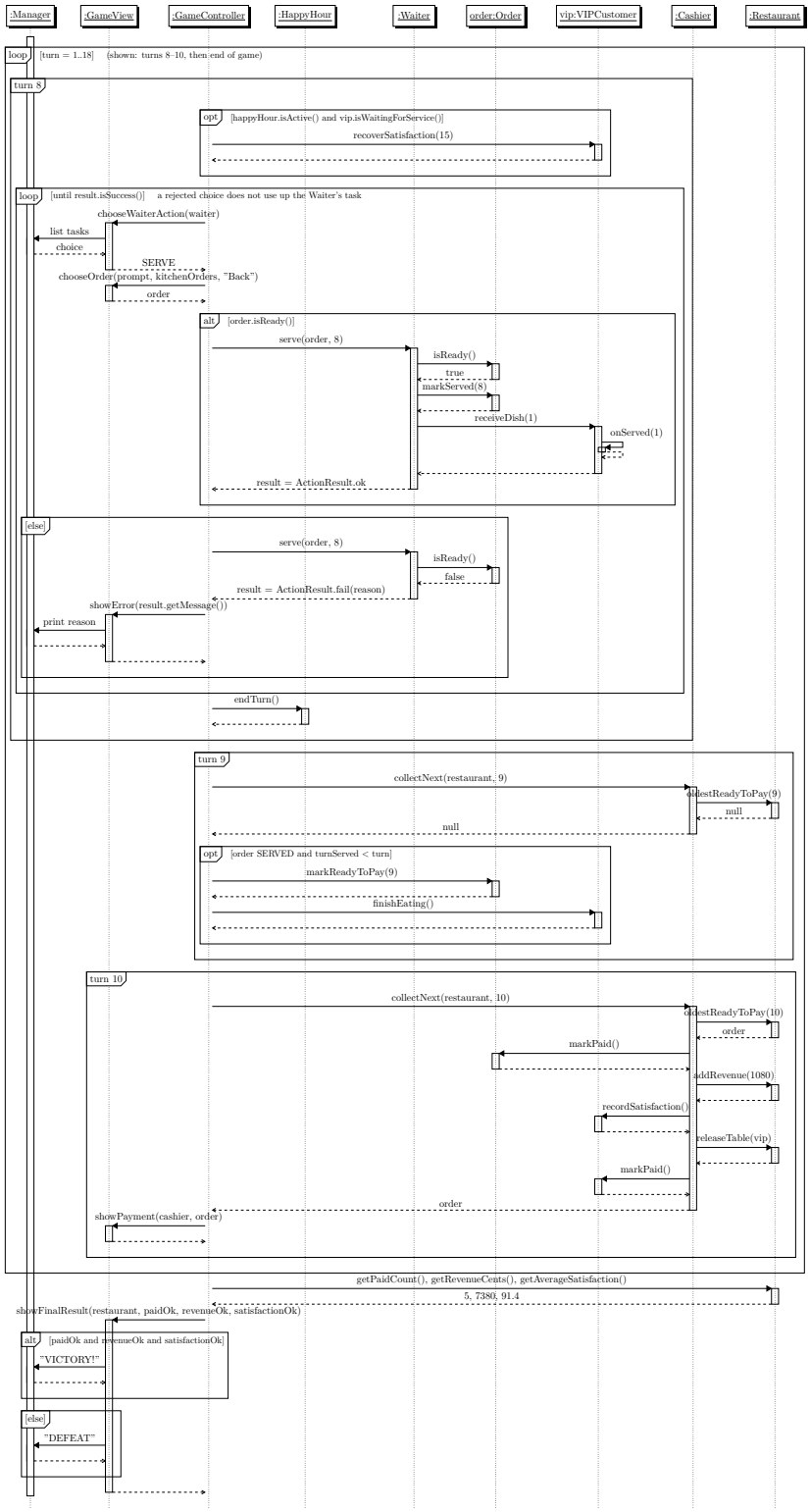


Figure 3: Sequence diagram, part 2 (turns 8–10 and the result check): Happy Hour recovery, serving with validation, eating, payment, and the end-of-game check. Each alt/[else] pair is one alternative fragment. The values returned by the result check are the real ones from this run.

not READY yet (0/1 units done)", the controller passes the message to `GameView.showError()`, and the inner loop asks the manager again. The Waiter's task for the turn is not used up.

3.4 How the Manager's Choice Changes the Outcome

To see what the Happy Hour decision does, we replayed the same game (seed 1) with the same staff choices, except that the manager answers "n" to Happy Hour. Table 4 shows the result.

	Happy Hour on turn 7	**No Happy Hour**
VIP's locked price	\$10.80	\$13.50
VIP's satisfaction recorded at payment (turn 10)	100	95
Final paid customers	5	5
Final revenue	\$73.80	\$76.50
Final average satisfaction (paid)	91.4	90.4
Result	Victory	Victory

Table 4: The same seed-1 game with and without Happy Hour

Happy Hour costs \$2.70 of revenue on the VIP's order. In return the +15 at the start of turn 8 cancels the 5 points the VIP lost on turn 7, so the VIP pays at 100 instead of 95 and the average rises from 90.4 to 91.4. In this run both games are comfortable wins, so the decision trades revenue for satisfaction rather than turning a defeat into a victory.

The trade matters more when the restaurant is under pressure. Under the rules, a Regular customer who starts turn 7 at 16 and is not served drops to 8 and then to 0 by the end of turn 8, and walks out. With Happy Hour on turns 7 and 8 they go $16 \to 31 \to 23$ and $23 \to 38 \to 30$, and would only reach 0 at the end of turn 12. That gives the manager four more turns to serve them. Only five customers can pay at all, so keeping one of them can decide whether the 4-paid target is met.

The Host's seating choice matters too. Because the Host seated the VIP on arrival, the Waiter could take the order on turn 7, inside the Happy Hour window, and the VIP paid on turn 10, well before the turn 18 cut-off.

4 Design Discussion

4.1 Key Abstractions and Responsibilities

We split the classes into entity, staff, and control/boundary groups. Each class below has a job that can be stated in one line, which we used as a quick cohesion test while refactoring between stages.

Abstraction	Responsibility
Entity	
`Customer` (abstract)	Owns satisfaction (kept in $[0, 100]$), lifecycle state (`CustomerState`) and the satisfaction recorded at payment. Leaves the type-specific rules (loss per turn, discount, food choice, reaction to service) to subclasses.
`RegularCustomer`, `VIPCustomer`, `CriticCustomer`	One concrete rule set each. Only `GameConfig.createArrival()` knows which one is being created.
`OrderItem` (interface)	What the rest of the program needs to know about food: name, price in cents, preparation units.
`MenuItem` (enum), `ComboMeal`	The three fixed dishes; a combo composed of two `MenuItem`s that derives its own price (90%, nearest cent) and units (sum).
`Menu`	The single dishes (`getDishes()`) and, in Stage 3, every pair of dishes as a combo (`getCombos()`); `getItems()` returns both for display.
`Order`	Locks the price when the order is taken and tracks preparation progress, `OrderStatus`, and the turns on which it was served and became `READY_TO_PAY`.
`Table`, `WaitingQueue`	One customer per table; arrivals kept in FIFO order.
`Restaurant`	Aggregate state: tables, queue, customers, orders and revenue, plus the shared rules on that state (`seatAt()`, `depart()`, `oldestReadyToPay()`) and the statistics (paid count, average recorded satisfaction).
`HappyHour`	A once-per-game pricing policy with its own lifecycle: available, active for two turns, used.
Staff	
`Staff` (abstract)	Name, role and a one-line duty. `GameView` lists the staff on duty through this type without knowing the roles.
`Waiter`, `Chef`, `Cashier`, `Host`	Each checks the rules of its own task (seat, take an order or serve; prepare one unit; collect the oldest eligible bill; seat one extra customer) and returns an `ActionResult` (the Cashier returns the paid `Order` or `null`).
`ActionResult`	Success flag plus a message. A failed action carries the reason shown to the player and does not use up the staff task.
Control and boundary	
`GameConfig`	Base setting constants, the arrival schedule and the stage switches (which customer types arrive; whether Host, combos and Happy Hour exist).
`GameController`	Runs the six turn steps in order and asks the right staff object to act. It also keeps the rules that depend only on the turn number: when eating finishes and when Happy Hour recovery applies.
`GameView`	All console input and output. Collects the manager's choices and prints state; contains no game rules.
`RestaurantRush`	Entry point: reads stage and seed, creates the view and the controller.

Table 5: Key abstractions and their responsibilities

The main cohesion fix during development was moving staff behaviour out of `GameController`. In our first draft the controller seated customers, priced orders and collected payments itself, so it had a reason to change for almost every rule in the brief. After the split, "only a dish that is already `READY` can be served" lives in `Waiter.serve()`, the seating checks live in `Restaurant.seatAt()` (shared by the Waiter and the Host), and the controller only decides *when* each staff member acts. The rule that a dish finished this turn is served next turn at the earliest follows from that ordering: the Waiter is asked before the Chef.

4.2 Design Principles

We only list principles that we can point to in the code. Where a principle is applied partly, we say so.

Single Responsibility (SRP). `GameView` changes only if the console layout changes. The pricing order (customer discount first, then Happy Hour) is in one place, `Waiter.takeOrder()`, and the price is then fixed inside `Order`. The average satisfaction of paid customers is computed once, in `Restaurant.getAverageSatisfaction()`, and reused by the turn header, the end-of-turn summary and the final result check. The weakest class here is `GameController`: besides sequencing the turn, it still holds the eating and Happy Hour timing, so a change to either rule would touch it.

Open/Closed (OCP). Stage 2 added two customer types without editing `GameController`: `VIPCustomer` and `CriticCustomer` override `getSatisfactionLoss()`, `applyDiscount()`, `chooseItem()` and `onServed()`. The only rule code we changed was `GameConfig.createArrival()`, which decides who arrives on turns 7 and 13. We did have to open `Customer` once, to add the `onServed()` hook, because Stage 1 gave a customer no way to react to slow service; `Waiter.serve()` now passes the waiting time to it. `GameView` and `RestaurantRush` also gained a stage menu. In Stage 3, `ComboMeal` is a new `OrderItem`, and `Order`, `Chef` and `Cashier` were not changed at all because they already worked against the interface. The customer side needed one small edit: `Customer.pickDefault()` gained the 30% combo branch, guarded by `menu.hasCombos()` so Stage 1 and 2 games draw exactly the same random numbers as before. OCP is not complete everywhere: Happy Hour needed a new parameter on `Waiter.takeOrder()` and a new phase in `GameController`, and a fifth staff role would need the same.

Liskov Substitution (LSP). Every `Customer` subtype keeps the base-class contract: the loss per turn is positive, `applyDiscount()` never returns more than its input, `chooseItem()` always returns an item from the given `Menu` (the Critic picks from `getDishes()`, which is a subset), and satisfaction stays in $[0, 100]$ because subclasses can only change it through the protected `adjustSatisfaction()`, which clamps. This is why `GameController` can loop over all customers in the abandonment step without special cases. The same holds for a `ComboMeal` used wherever a `MenuItem` is expected.

Interface Segregation (ISP). `OrderItem` has three methods. `GameView` uses all three to print the menu, `Order` needs the preparation units, and `Waiter` needs the name and price. Splitting it further would give one-method interfaces with the same two implementers. We deliberately did not add interfaces such as `Seatable` or `Payable`: each would have had exactly one implementer, and the brief warns against interfaces created only to satisfy a checklist.

Dependency Inversion (DIP), partly applied. For the parts that vary, high-level code depends on abstractions: `GameController`, the staff classes and `GameView` work with `Customer`, `OrderItem` and `Staff`. The concrete customer classes are named only in `GameConfig.createArrival()`, and `ComboMeal` is constructed only in `Menu`. We did not invert the staff dependency: `GameController` constructs `Waiter`, `Chef`, `Cashier` and `Host` itself. Each role has one implementation and a different action method, so a shared "perform action" interface would have been artificial.

4.3 Extensibility Walkthrough

Change	What has to be touched
New customer type (e.g. a "Family" that loses 4 per turn)	One new subclass of `Customer`; one line in `GameConfig.createArrival()`. Controller, staff and view unchanged.
New menu item (e.g. Soup, $6, 1 unit)	One enum constant in `MenuItem`. Combos including Soup appear automatically because `Menu` builds the pairs from `MenuItem.values()`, and the Critic considers it automatically.
New pricing policy (e.g. a lunch discount)	A class like `HappyHour`, a parameter on `Waiter.takeOrder()`, and an activation step in `GameController`. If several policies were needed we would introduce a `PricingPolicy` interface at that point, not before.
Different base setting (more tables, longer game, other targets)	Constants in `GameConfig` only.

Table 6: Cost of likely future changes

4.4 Trade-offs

Decision	Why we chose it	What it costs
`ActionResult` instead of exceptions for invalid choices	An invalid choice is a normal part of play, not an error. The view prints the reason and asks the same staff member again, so the task is not used up.	Every caller must check `isSuccess()`; in our code the three choice loops in `GameController` do this.
`Restaurant` aggregate	One object to pass to staff methods instead of four lists; seating and payment rules sit next to the data they check.	`Restaurant` is a larger class, and every staff method can see all state.
`MenuItem` as an enum	Fixed set of dishes, safe identity comparison, `values()` for building combos.	Dishes cannot be loaded from a file at run time without changing the type.
Stage switches in `GameConfig`	The final build can run all three stages for testing and the video, and the stage differences are in one place.	The controller still has `host != null` and `happyHour != null` checks. The tags `stage-1` and `stage-2` do not contain this code at all.
Controller still sequences the steps	The six steps must run in a fixed order; keeping that order in one short method (`playTurn()`) matches the brief and is easy to debug.	`GameController` is the class most likely to change when a timing rule changes.

Table 7: Main design trade-offs

4.5 Alternative Design Considered: State Pattern for the Customer Lifecycle

We considered replacing `CustomerState` and `OrderStatus` with State objects (`WaitingState`, `SeatedState`, `EatingState`, …), each deciding whether satisfaction drops and which transition comes next. This would remove state checks such as `Customer.isWaitingForService()` and the status tests in `Waiter.serve()` and `Chef.prepare()`.

We rejected it for three reasons. First, the lifecycle is a straight line with one exit (leaving at 0 satisfaction), and the guards we already have reject every illegal move: `Waiter.serve()` only accepts a `READY` order, `Chef.prepare()` only a `PLACED` or `COOKING` one, and `Restaurant.oldestReadyToPay()` only returns an order that became `READY_TO_PAY` on an

earlier turn. Second, the variation the brief actually asks for is *between customer types*, not between states, and subclassing `Customer` already handles it. Combining both would mean every State class asks the customer for its type-specific numbers, which adds indirection without new behaviour. Third, the timing rules ("served on an earlier turn", "ready to pay on an earlier turn") compare turn numbers stored in `Order`, which is simpler to check in one place than across seven State classes. If a later stage added branching lifecycles, for example take-away orders that skip the table, we would revisit this.

4.6 How the Player's Choices Change the Result

The Waiter has one task per turn, so most turns force a real choice. We replayed the following case in the Stage 1 build (seed 42), once for each option, and the numbers below are from those runs.

At the start of turn 4, Regular #1's Pasta is `READY` (satisfaction 76), Regular #2 has just arrived (100), and Table 2 is free.

- **Serve first.** Regular #1 stops losing satisfaction, finishes eating on turn 5 and pays $18.00 on turn 6 with 76 recorded. Regular #2 stays in the queue and ends turn 4 at 92; it can be seated on turn 5.
- **Seat first.** Regular #2 also ends turn 4 at 92, because being seated does not stop the loss; only being served does. Regular #1 drops to 68, is served on turn 5 and pays on turn 7 with 68 recorded, so Table 1 is freed one turn later.

Seating first gains Regular #2 one turn (the order can be taken on turn 5 instead of 6), but the 8 points Regular #1 loses are recorded at payment and stay in the average for the rest of the game. With four paid customers, that single decision lowers the final average by 2 points, so a manager who repeatedly delays ready dishes eats into the margin above the target of 60.

Other choices with a measurable effect:

- **Chef priority.** A Pasta needs two Chef turns. Cooking a newly ordered Salad first finishes that order a turn earlier, but the Pasta customer loses 8 (Regular), 5 (VIP) or 12 (Critic) for every extra turn.
- **Critic timing.** The Critic always orders the most expensive single dish, Pasta. Served within two turns of ordering it gains 5; three or more turns costs 10, a 15-point swing on one customer. The bonus is only reachable if the Chef cooks the Pasta on the turn it is ordered and on the next turn, and the Waiter serves it on the turn after that.
- **Happy Hour timing.** On each of its two turns, every customer still waiting for a table or for food gets 15 back (capped at 100). This includes seated customers whose dish has not been served, so with two such customers it can restore up to 60 points. Customers who are eating, ready to pay or have paid get nothing. Activated when nobody is waiting for service, it only gives the 20% discount, which lowers revenue and makes the $45 target harder.

5 GitHub Evidence and Stage Transitions

5.1 Repository and Tags

All work is kept in one repository:

https://github.com/SeraphinaXYX/SC2002-Restaurant-Rush

Each stage is marked with a tag on a commit that compiles and runs on its own (`javac -d bin src/restaurantrush/*.java`). Checking out a tag gives exactly the code that existed at the

end of that stage, together with the input scripts and recorded console output for that stage's runs in `test-runs/`.

Tag	What the snapshot contains
`stage-1`	20 source files: Regular customers, the three menu items, `Waiter`, `Chef`, `Cashier`, `Table`, `Order`, `WaitingQueue`, `Restaurant`, `ActionResult`, the full six-step turn, abandonment, payment and both endings. Searching the snapshot for "VIP", "Critic", "Host", "combo" and "Happy Hour" finds nothing (the only hits are the words "unhappy departures" in the end-of-turn display).
`stage-2`	22 source files: everything in `stage-1` plus `VIPCustomer` and `CriticCustomer`. The arrivals on turns 7 and 13 become a VIP and a Critic, so there are still six arrivals, and the game asks which stage to play.
`stage-3`	25 source files: everything in `stage-2` plus `Host`, `ComboMeal` and `HappyHour`. This is the build the class diagram in Section 2 describes.

Table 8: Contents of each tagged snapshot

The tables below were produced from `git diff --numstat` between the tags, so they match what the compare links show. Changes to `README.md`, `TESTING.md` and `test-runs/` are left out of the tables; they only document the new stage.

5.2 Stage 1 → Stage 2

Compare link:

https://github.com/SeraphinaXYX/SC2002-Restaurant-Rush/compare/stage-1...stage-2

Change	File	+/−	Why it was needed
New	`VIPCustomer.java`	38/0	Overrides `getSatisfactionLoss()` (5) and `applyDiscount()` (10% off).
New	`CriticCustomer.java`	57/0	Loss of 12, always picks the most expensive dish, and reacts to service speed in `onServed()`.
Changed	`GameConfig.java`	19/3	Stores the chosen stage; `createArrival()` returns a VIP on turn 7 and a Critic on turn 13 from Stage 2.
Changed	`Customer.java`	9/1	Adds the empty `onServed(int waitTurns)` hook, called from `receiveDish()`, so the Critic can react without the controller knowing it is a Critic.
Changed	`Waiter.java`	9/2	`serve()` passes the wait (serve turn minus order turn) to `receiveDish()` and reports any reaction in its message.
Changed	`GameView.java`	23/1	New `askStage()` prompt; the banner shows the stage.
Changed	`RestaurantRush.java`	2/1	Asks for the stage and passes it to `GameConfig`.
Deleted	none	–	No files were removed.

Table 9: Source files added and changed from Stage 1 to Stage 2 (157 lines added, 8 removed)

Most of this is the kind of small registration and setup change the brief says to expect: `GameConfig`, `GameView` and `RestaurantRush` only learn that a stage number exists and which customer arrives when. The one change to an existing rule class is the `onServed()` hook in `Customer` and the matching call in `Waiter.serve()`. We added it because the Critic's reaction depends on how long the food took, and the cleanest place to find that out is the moment the dish is served. It is a no-op for Regular and VIP customers, so their behaviour is unchanged. `GameController.java` is

not in the diff at all: it still calls `getSatisfactionLoss()`, `applyDiscount()` and `chooseItem()` on whatever `Customer` it holds, and the subclass decides what happens.

5.3 Stage 2 → Stage 3

Compare link:

https://github.com/SeraphinaXYX/SC2002-Restaurant-Rush/compare/stage-2...stage-3

Change	File	+/−	Why it was needed
New	`Host.java`	33/0	A new `Staff` subclass whose `seat()` returns an `ActionResult`, like the Waiter's.
New	`ComboMeal.java`	43/0	Implements `OrderItem`; two `MenuItems`, prep units summed, price 90% of the sum rounded to the nearest cent.
New	`HappyHour.java`	53/0	The one-off pricing policy: used or not, turns remaining, the 20% discount and the +15 recovery amount.
Changed	`Menu.java`	31/2	`Menu(boolean includeCombos)` builds the six combos; adds `getCombos()`, `hasCombos()` and `getItems()`.
Changed	`Customer.java`	14/1	`pickDefault()` gives a 30% seeded chance of a combo when the menu has combos; adds `recoverSatisfaction()`.
Changed	`CriticCustomer.java`	2/1	Comment only: explains that a Critic never orders a combo.
Changed	`Waiter.java`	10/3	`takeOrder()` takes the `HappyHour` (null below Stage 3) and applies it after the customer's own discount, before the price is locked into the `Order`.
Changed	`GameConfig.java`	12/0	`hasHost()`, `hasHappyHour()` and `hasCombos()`, so each stage switches on only its own features.
Changed	`GameController.java`	83/4	Creates the Host and Happy Hour for Stage 3, adds `happyHourStart()` and `hostTurn()`, and calls `happyHour.endTurn()` at the end of the turn.
Changed	`GameView.java`	33/2	Stage 3 menu line, Happy Hour prompt and status line (turns left), recovery message, combos in the menu list.
Changed	`RestaurantRush.java`	1/1	`askStage(3)` instead of `askStage(2)`.
Deleted	none	–	No files were removed.

Table 10: Source files added and changed from Stage 2 to Stage 3 (315 lines added, 14 removed)

Again, `GameConfig`, `RestaurantRush`, the `Menu` constructor flag and the new view prompts are setup changes. The real rule changes are three, and each one is something Stage 3 asks for:

- **`GameController`** gained two new phases. The six steps keep their order; Stage 3 only inserts the Happy Hour step at the start of the turn and the Host step just before the Waiter. The two new methods sit next to the existing step methods instead of being mixed into them.
- **`Waiter.takeOrder()`** is where the price is locked, so it is the only correct place to apply Happy Hour. The customer discount is still applied first, which gives the brief's example of $\$15 \times 0.9 \times 0.8 = \$10.80$.
- **`Customer.pickDefault()`** needed the combo rule. The combo branch is only entered when `menu.hasCombos()` is true, so in Stages 1 and 2 the seeded random generator is called exactly as before and the same seed still gives the same dishes. The replay check below confirms this.

5.4 What Stayed the Same, and How We Checked

Thirteen of the twenty Stage 1 source files are byte-for-byte identical at `stage-3`: `ActionResult`, `Cashier`, `Chef`, `CustomerState`, `MenuItem`, `Order`, `OrderItem`, `OrderStatus`, `RegularCustomer`, `Restaurant`, `Staff`, `Table` and `WaitingQueue`. `VIPCustomer` is also unchanged from Stage 2 to Stage 3. So the order lifecycle, the Cashier's timing (one payment per turn, oldest first, never on the turn a customer becomes `READY_TO_PAY`), satisfaction loss, abandonment and the victory check were not edited after Stage 1.

Not editing a file does not prove the behaviour is unchanged, because the controller calls it differently, so we also replayed the old runs on the final build. We extracted the `stage-3` snapshot into a clean folder, compiled it, and fed it each Stage 1 input script with `--echo`, choosing Stage 1 and seed 42. The scripts at `stage-3` are the `stage-1` scripts with one extra first line (`1`) to answer the stage menu, which did not exist in Stage 1. We then compared the output with the transcript recorded at the `stage-1` tag, ignoring only the stage menu and the banner line that now names the stage. We did the same for the Stage 2 scripts against the `stage-2` recordings.

Script	Recorded at	Replayed on	Result
`stage1-victory`	`stage-1`	`stage-3`	Identical. Victory: 5 paid, $63.00, average 74.4.
`stage1-defeat`	`stage-1`	`stage-3`	Identical. Defeat: 0 paid, $0.00, 2 unhappy departures.
`stage1-invalid-input`	`stage-1`	`stage-3`	Identical, including every "explain and ask again" message.
`stage2-victory`	`stage-2`	`stage-3`	Identical, including the VIP's discounted bill and the Critic's reaction.
`stage2-defeat`	`stage-2`	`stage-3`	Identical.

Table 11: Replaying earlier runs on the final build

Every turn of these games, including the customers' choices of food, came out the same, so the Stage 3 changes did not leak into the earlier stages. The exact commands are in `TESTING.md`.

5.5 How the Stage 1 Design Took the Later Requirements

We did not know the exact Stage 2 and Stage 3 rules when we first sketched the classes, but we did know that customer types and menu items were the parts most likely to grow. Three early decisions paid off. Making `Customer` abstract meant VIP and Critic were two new subclasses, and `GameController` did not change at all in Stage 2. Having `Order` depend on the `OrderItem` interface rather than on `MenuItem` meant a `ComboMeal` could be ordered, cooked and paid for with no change to `Order`, `Chef` or `Cashier`. And because every staff action returns an `ActionResult`, the `Host` reuses the same "explain why it failed and ask again" handling as the Waiter instead of needing its own error path.

The place that did grow is `GameController`, by 83 lines in Stage 3, since it has to schedule the new phases. We accepted this because the turn order belongs in one place and the new phases are separate methods. We also missed one thing: the `onServed()` hook was not in our Stage 1 `Customer`, so Stage 2 had to add it. If we had listed "events a customer might react to" when designing Stage 1, that change would not have been needed.

6 Functional Evidence

Everything in this section is copied from transcripts produced by the submitted Stage 3 build. We did not edit the console output, apart from cutting long stretches, which are marked with `[...]`. Each run is stored in the repository under `test-runs/` as a pair of files. The `.txt` file holds the exact keyboard input: the stage, the seed, then one line per choice. The `.out` file is the transcript we recorded. A TA can replay any run with:

```
javac -d bin src/restaurantrush/*.java
java -cp bin restaurantrush.RestaurantRush --echo < test-runs/stage1-victory.txt
```

`--echo` prints each input after its prompt, so the replay reads like a live game. If an input file ends early, every remaining prompt takes its last option (Wait, Back or "n"), so the game still plays all 18 turns. `TESTING.md` in the repository describes each run in more detail.

6.1 Summary of reproducible runs

All runs use the base setting from the brief: 18 turns, 2 tables, one Waiter, one Chef and one Cashier, with arrivals on turns 1, 4, 7, 10, 13 and 16. Stage 3 adds the Host. The "victory" runs follow one simple rule each turn. The Waiter serves a READY dish if there is one; otherwise takes an order; otherwise seats someone. The Chef always works on the oldest unfinished order. The "defeat" runs use the same Waiter, but the Chef never cooks.

Table 12: Reproducible runs (all numbers taken from the final screen of each `.out` file)

Stage	Seed	Input file (`test-runs/`)	Result	Paid	Revenue	Avg sat.
1	42	`stage1-victory.txt`	Victory	5	$63.00	74.4
1	42	`stage1-defeat.txt`	Defeat	0	$0.00	N/A
1	42	`stage1-invalid-input.txt`	Defeat	1	$18.00	76.0
2	42	`stage2-victory.txt`	Victory	4	$53.10	76.3
2	42	`stage2-defeat.txt`	Defeat	0	$0.00	N/A
2	42	`extra-critic-slow-service.txt`	Victory	4	$53.10	76.3
3	42	`stage3-victory.txt`	Victory	5	$88.45	83.2
3	42	`stage3-defeat.txt`	Defeat	0	$0.00	N/A
3	42	`stage3-happy-hour-after-payment.txt`	Victory	5	$88.74	87.6
3	1	`stage3-vip-happy-hour-seed1.txt`	Victory	5	$73.80	91.4
3	42	`extra-host-and-waiter-seat.txt`	Defeat	0	$0.00	N/A

The `invalid-input` and `extra-*` runs exist to show one specific behaviour. They stop giving real choices after a few turns, so their final results are not meaningful. No victory run has more than five paid customers. This matches the note in the brief: the customer who arrives on turn 16 cannot pay within 18 turns.

6.2 Start-of-turn display

The brief asks for the turn number, revenue, the average satisfaction of paid customers, the queue, the occupied tables, each customer's satisfaction, order status, and the staff tasks. All of these appear at the top of every turn. In Stage 3, the Happy Hour status and the Host are also shown. Any Happy Hour recovery is applied first, so the table shows the satisfaction after the +15 (`stage3-victory.out`, turn 8):

```
======================================================================
 TURN 8 / 18     Revenue: $9.00     Avg satisfaction (paid): 92.0
```

```
======================================================================
  * Happy Hour: VIP #3 cheers up, satisfaction 95 -> 100.
 Happy Hour: ACTIVE, 1 turn(s) left including this one (20% off new orders, waiting customers
     +15)
 Waiting queue:
   (empty)
 Tables:
   Table 1: VIP #3 [satisfaction 100, ORDERED]
            order: COOKING 1/2 units, $15.55
   Table 2: Regular #2 [satisfaction 84, READY_TO_PAY]
            order: READY_TO_PAY 2/2 units, $21.60
 Staff on duty:
   Host Hana         seat one waiting customer (in addition to the Waiter)
   Waiter Wendy      one task: seat, take an order or serve a READY dish
   Chef Carlos       one preparation unit on one order
   Cashier Cathy     automatic, collects one payment per turn
```

At the end of each turn, the game prints the served count, unhappy departures, revenue, average satisfaction and turns used:

```
End of turn 8: served 2 | unhappy departures 0 | revenue $30.60
                avg satisfaction (paid) 88.0 | turns used 8/18
```

6.3 Invalid choices are explained and do not use up the task

In `stage1-invalid-input.out`, the manager types text, then an out-of-range number, then two actions that are impossible on turn 1. The Waiter's menu reappears every time, and the turn continues only after a valid action:

```
Waiter Wendy - choose one task:
  1. Seat a waiting customer
  2. Take an order
  3. Serve a READY dish
  4. Wait
Choice > abc
 ! Please enter a number from 1 to 4.
Choice > 9
 ! Please enter a number from 1 to 4.
Choice > 3
 ! Not possible: There are no orders in the kitchen. Choose again.
Waiter Wendy - choose one task:
  [...]
Choice > 2
 ! Not possible: No customer is seated yet. Choose again.
```

The reasons come from the `ActionResult` returned by `Waiter`, not from the view. The same run also tries to serve a half-cooked Pasta and to seat a customer at an occupied table:

```
Serve which dish?
  1. Pasta for Regular #1 - COOKING 1/2 units, $18.00
  2. Back
Choice > 1
 ! Not possible: Pasta for Regular #1 is not READY yet (1/2 units done). Choose again.
[...]
 ! Not possible: Table 1 is occupied by Regular #1. Choose again.
```

6.4 READY, SERVED, READY_TO_PAY, PAID: the worked example from the brief

The first six turns of `stage1-victory.out` repeat the example in Section 3.3 of the brief. Regular #1 orders Pasta (2 units) and gets the manager's full attention. The customer's satisfaction at the end of turns 1–3 is 92, 84 and 76. It then stays at 76 once the dish is served, and 76 is the value recorded at payment.

```
TURN 2  * Waiter Wendy takes Regular #1's order: Pasta ($18.00).
        * Chef Carlos works on Pasta for Regular #1 (1/2).
TURN 3  * Chef Carlos works on Pasta for Regular #1 (2/2) - READY, can be served from next turn
    .
TURN 4    Table 1: Regular #1 [satisfaction 76, ORDERED]
                  order: READY 2/2 units, $18.00
        * Waiter Wendy serves Pasta for Regular #1.
TURN 5    Table 1: Regular #1 [satisfaction 76, EATING]
        * Regular #1 finishes eating and is READY_TO_PAY.
TURN 6  * Cashier Cathy collects $18.00 from Regular #1 (satisfaction recorded: 76). Table
    freed.
```

To save space, the excerpt above shows only the relevant lines of turns 2–6; each line is copied word for word. The ordering rules can all be seen here. The Chef cooks an order the Waiter took in the same turn (turn 2). A dish that becomes READY on turn 3 cannot be served until turn 4. A customer who becomes READY_TO_PAY on turn 5 pays on turn 6, at the earliest.

6.5 Stage 2: VIP and Critic behaviour

VIP pricing. VIP #3 orders a Salad, and the 10% discount is locked into the order when it is taken (`stage2-victory.out`):

```
  * Waiter Wendy takes VIP #3's order: Salad (list $9.00, charged $8.10).
  [...]
  * Cashier Cathy collects $8.10 from VIP #3 (satisfaction recorded: 85). Table freed.
```

The VIP's satisfaction also drops more slowly: VIP #3 lost 5 points per turn and was recorded at 85, while Regular #1 and Regular #2 in the same run lost 8 per turn and were recorded at 76.

Critic preference and reaction. We documented that a Critic always orders the most expensive single dish (Pasta). A Critic gains 5 satisfaction if served within 2 turns of ordering and loses 10 if it takes 3 or more turns. Both cases appear in the transcripts:

```
stage2-victory.out (ordered turn 15, served turn 17):
  * Waiter Wendy takes Critic #5's order: Pasta ($18.00).
  [...]
  * Waiter Wendy serves Pasta for Critic #5. Critic #5 reacts: satisfaction 52 -> 57.

extra-critic-slow-service.out (same game, Waiter waits on turn 17, serves on turn 18):
  * Waiter Wendy serves Pasta for Critic #5. Critic #5 reacts: satisfaction 40 -> 30.
```

The Critic arrives on turn 13 and is served too late to pay, so the slow-service run ends with the same result as the normal Stage 2 victory run. The Critic also loses satisfaction faster (12 per turn): it arrived at 100 and was at 52 four turns later, when the Pasta reached the table.

6.6 Stage 3: Host, ComboMeal and Happy Hour

Host in addition to the Waiter. In `extra-host-and-waiter-seat.out`, two customers are queuing on turn 4. The Host seats one, and the Waiter seats the other in the same turn:

```
Host Hana - seat which customer?
  1. Regular #1 [satisfaction 76, WAITING]
  2. Regular #2 [satisfaction 100, WAITING]
  3. Wait
Choice > 1
[...]
 * Host Hana seats Regular #1 at Table 1.
[...]
 * Waiter Wendy seats Regular #2 at Table 2.
```

ComboMeal. The Stage 3 menu lists all six combos. Each combo's price is 90% of its two parts, rounded to the nearest cent, and its preparation units are the sum of its two parts. For example, $(\$9 + \$18) \times 0.9 = \$24.30$, with $1 + 2 = 3$ units. Customers in Stage 3 order a random combo 30% of the time, using the seeded generator.

```
  Salad + Salad Combo          $16.20   2 prep unit(s)
  Salad + Burger Combo         $21.60   2 prep unit(s)
  Salad + Pasta Combo          $24.30   3 prep unit(s)
  Burger + Burger Combo        $27.00   2 prep unit(s)
  Burger + Pasta Combo         $29.70   3 prep unit(s)
  Pasta + Pasta Combo          $32.40   4 prep unit(s)
  [...]
  * Waiter Wendy takes Regular #4's order: Salad + Pasta Combo ($24.30).
```

Happy Hour. The manager switches it on at the start of turn 7, before any staff act. It lasts for that turn and the next. The remaining duration is printed at the top of both turns: "2 turn(s) left including this one" on turn 7 and "1 turn(s) left" on turn 8 (see the start-of-turn display above). With seed 1, the VIP orders a Burger during Happy Hour. The VIP discount is applied first and Happy Hour second, giving exactly the brief's example of $\$15 \times 0.9 \times 0.8 = \$10.80$:

```
stage3-vip-happy-hour-seed1.out:
  * Waiter Wendy takes VIP #3's order: Burger (list $15.00, charged $10.80).

stage3-victory.out (combo for a VIP: $21.60 x 0.9 = $19.44, x 0.8 = $15.552 -> $15.55):
  * Happy Hour is ON for this turn and the next: 20% off new orders.
  [...]
  * Waiter Wendy takes VIP #3's order: Salad + Burger Combo (list $21.60, charged $15.55).
```

`stage3-happy-hour-after-payment.out` checks the recovery rule. Happy Hour is switched on at turn 13. By then three customers have already paid, with recorded satisfaction 92, 84 and 90 (average 88.7). Only customers still waiting for a table or for food recover 15 points, on the activation turn and on the following turn. The paid average stays at 88.7 throughout (only the relevant lines of turns 13–15 are shown):

```
 TURN 13 / 18     Revenue: $50.04     Avg satisfaction (paid): 88.7
  * Happy Hour is ON for this turn and the next: 20% off new orders.
  * Happy Hour: Regular #4 cheers up, satisfaction 76 -> 91.
 Happy Hour: ACTIVE, 2 turn(s) left including this one (20% off new orders, waiting customers
     +15)
 TURN 14 / 18     Revenue: $50.04     Avg satisfaction (paid): 88.7
  * Happy Hour: Critic #5 cheers up, satisfaction 88 -> 100.
 Happy Hour: ACTIVE, 1 turn(s) left including this one (20% off new orders, waiting customers
     +15)
 TURN 15 / 18     Revenue: $50.04     Avg satisfaction (paid): 88.7
 Happy Hour: used
```

The final average of 87.6 equals $(92 + 84 + 90 + 91 + 81)/5$. These are the values recorded at each payment, so Happy Hour never changes a customer who has already paid. Orders placed before turn 13 keep their original prices. For example, Regular #4's combo was ordered at \$24.30 and was still charged \$24.30.

6.7 Abandonment, Victory and Defeat

In the defeat runs the Chef never cooks. Satisfaction keeps falling by 8 per turn, and Regular #1 leaves on turn 13 without paying. The leaving customer's order is cancelled and their table is freed (`stage1-defeat.out`):

```
  * Chef Carlos waits this turn.
  * Regular #1 runs out of patience and leaves without paying.
--------------------------------------------------------------------------------
 End of turn 13: served 0 | unhappy departures 1 | revenue $0.00
                 avg satisfaction (paid) N/A | turns used 13/18
```

The game always plays all 18 turns and then checks the three targets:

```
stage1-victory.out:
  VICTORY! The restaurant hit every target.
  Paid customers    5        (need 4)    [met]
  Revenue           $63.00   (need $45.00)  [met]
  Avg satisfaction  74.4     (need 60)   [met]
  Unhappy departures: 0

stage1-defeat.out:
  DEFEAT. At least one target was missed.
  Paid customers    0        (need 4)    [missed]
  Revenue           $0.00    (need $45.00)  [missed]
  Avg satisfaction  N/A      (need 60)   [missed]
  Unhappy departures: 2
```

The turn-16 customer. The game says at the start that the customer arriving on turn 16 cannot finish paying in time. In `stage1-victory.out`, the best case for that customer is an order taken on turn 18 and a dish that becomes READY on the same turn. The game then ends with 5 of the 6 customers paid, as the brief expects:

```
(The customer arriving on turn 16 cannot finish paying in time - that is expected.)
[...]
  * Waiter Wendy takes Regular #6's order: Burger ($15.00).
  [...]
  * Chef Carlos works on Burger for Regular #6 (1/1) - READY, can be served from next turn.
```

7 Reflections

7.1 Who Did What

Member	Main ownership
Member A	`GameController` and `GameConfig`: the six-step turn loop, arrival schedule, result check
Member B	`Customer`, `CustomerState`, `Order`, `OrderStatus`, `Table`, `WaitingQueue`, `Restaurant`
Member C	`Staff`, `Waiter`, `Chef`, `Cashier`, `ActionResult`; Stage 2 `VIPCustomer` and `CriticCustomer`
Member D	Stage 3: `Host`, `ComboMeal`, `Menu`, `HappyHour`; pricing checks
Member E	`GameView` and `RestaurantRush`, UML diagrams, test runs, report editing, video

Table 13: Ownership by member

Before each stage tag, every member read through at least one other member's classes, so each of us can explain code we did not write.

7.2 Problems We Hit

Payment timing. Our first Stage 1 build let a customer pay on the same turn they finished eating, so the Pasta example in the brief paid on turn 5 instead of turn 6. We fixed it by ordering step 4 so that the `Cashier` collects first, and only orders that were already `READY_TO_PAY` can be charged. Only after that do `SERVED` orders from earlier turns move to `READY_TO_PAY`. A dish served on turn 4 now becomes `READY_TO_PAY` on turn 5 and is paid on turn 6, which is what `stage1-victory.out` shows (Section 6).

Staff logic in the controller. At first all seating, ordering and serving rules lived inside `GameController`. It worked, but when we drew the class diagram the controller was one huge box and we could not say who was responsible for what. We moved each task into `Waiter`, `Chef`, `Cashier` and later `Host`. Every action now returns an `ActionResult`, so an invalid choice, such as

serving a Pasta that is still 1/2 prepared, prints the reason and the manager picks again without losing the staff task.

Happy Hour and paid customers. In an early Stage 3 version, the +15 recovery was given to everyone who no longer had a table. That included customers who had already paid, so their recorded satisfaction went up and the average looked better than it really was. We only noticed because one test run ended with an average of 99. The recovery now checks the customer's state, so only customers still waiting for a table or for food recover, and the run `stage3-happy-hour-after-payment` keeps the paid average unchanged through both Happy Hour turns.

What the screen shows on Happy Hour turns. In the peer review before the final tag, a teammate pointed out that the start-of-turn display was printed before the recovery was applied. The numbers on screen were therefore 15 lower than the ones the turn actually used. We moved the recovery before the state display, so what the manager sees matches the rules.

The Critic and combo meals. Our first idea was that a Critic orders the most expensive item on the whole menu. Once combos existed, that item became Pasta + Pasta, which needs 4 preparation units. Counting it through, a Critic arriving on turn 13 would be served on turn 18 at the earliest and could never pay. We changed the rule so that a Critic judges the kitchen on its most expensive single dish (Pasta). This keeps the Critic's preference distinct while leaving the Stage 2 behaviour unchanged.

The turn-16 customer. During testing we first thought the last customer never paying was a bug. Tracing it by hand showed that even with perfect play they cannot reach `READY_TO_PAY` in time, which the brief says is intended. We plan around at most five payers.

Money rounding. Prices are stored as `int` cents, and rounding happens after each step, with the final price locked into the `Order`. A Salad + Pasta combo is $2700 \times 0.9 = 2430$ cents ($24.30). A VIP Burger in Happy Hour is $1500 \times 0.9 = 1350$, then $1350 \times 0.8 = 1080$ cents ($10.80). Both match the brief.

7.3 What We Would Do Differently

We would draw the class diagram before writing Stage 1 code, since the controller problem only became obvious on paper. We would also add JUnit tests for pricing and turn timing early on. Most of our checking was done by replaying seeded runs and comparing transcripts, which caught real bugs but was slow to set up.

7.4 What We Learned

Polymorphism paid off most when the requirements changed. Adding VIP and Critic customers needed two new subclasses, a small hook in `Customer`, a change to `Waiter.serve()` so it passes the waiting time, and the arrival switch in `GameConfig`. `GameController` had no diff at all between `stage-1` and `stage-2`. In Stage 3, `ComboMeal` implementing `OrderItem` meant `Order`, `Chef` and `Cashier` stayed untouched. Happy Hour was the opposite case: it is a rule about the whole turn, so it did need changes to `GameController` and `Waiter.takeOrder()`. Our main lesson is that small classes with one clear job were easier to extend, test and explain than one class that does everything. Replaying the Stage 1 and Stage 2 scripts on the final build and getting the same transcripts gave us confidence that the later stages did not break the earlier rules.

8 Video Timestamps

The video is about 15 minutes long. Every member presents at least one segment, using the submitted project and the replay files in `test-runs/`.

#	Timestamp	Segment	Presenter
1	TBD	The game launching from the submitted project	Member E
2	TBD	Meaningful manager choices, including a decision where staff are constrained	Member A
3	TBD	A dish going `READY` → `SERVED` → payment	Member B
4	TBD	A VIP or Critic behaviour	Member C
5	TBD	A Stage 3 feature (Host, ComboMeal, Happy Hour)	Member D
6	TBD	One victory and one defeat run	Member A
7	TBD	Stage tag comparisons and the diagrams	Member E

Table 14: Video segments

The recorded video, or a download link that stays available to graders, is included in the submission archive. The same timestamps are listed in the README.